

DDA5001 Machine Learning

Linear Classification II: Logistic Regression

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Recap: VC Dimension Generalization Result

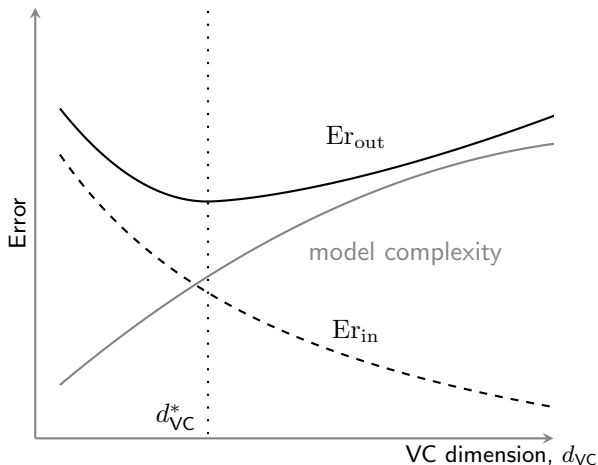
VC generalization bound

For any $\delta > 0$, with probability at least $1 - \delta$, we have the following generalization bound:

$$\forall f \in \mathcal{H} \quad \text{Er}_{\text{out}}(f) \leq \text{Er}_{\text{in}}(f) + \mathcal{O} \left(\sqrt{\frac{d_{\text{VC}}}{n}} \right)$$

- ▶ This result is very general to cover all cases, and hence it is a **loose** result.
- ▶ It still provides meaning information about learning. For instance, more training data is always better and larger d_{VC} has a worse generalization ability.

Recap: Learning Curve from VC Analysis



- The **optimal model** is the one that minimizes the combinations of Er_{in} and generalization error.

Agenda

Logistic Regression

Classification Methodology II: Logistic Regression

Conditional Probability for Classification

- ▶ We are going to classify 'Approve' and 'Reject'.
- ▶ Labeling: 'Approve' $y = +1$, 'Reject' $y = -1$.
- ▶ Now you have a test data x without labeling



- ▶ Suppose now you know

$$\Pr[y = +1|x] = 0.8, \quad \Pr[y = -1|x] = 0.2$$

Which class you will assign x to?

Optimal Classifier Induced by Conditional Probability

Bayes-optimal classifier

The classifier

$$y \leftarrow \operatorname{argmax}_{y \in \mathcal{Y}} \Pr[y|\mathbf{x}]$$

is optimal over all possible classifiers.

- ▶ $\Pr[y|\mathbf{x}]$ is called **a-posteriori probability** of y .
- ▶ Implication: **Compute $\Pr[y|\mathbf{x}]$ for optimal classification.**

Optimal Classifier Induced by Conditional Probability

Bayes-optimal classifier

The classifier

$$y \leftarrow \operatorname{argmax}_{y \in \mathcal{Y}} \Pr[y|\mathbf{x}]$$

is optimal over all possible classifiers.

- ▶ $\Pr[y|\mathbf{x}]$ is called **a-posteriori probability** of y .
- ▶ Implication: **Compute $\Pr[y|\mathbf{x}]$ for optimal classification.**
- ▶ How can we know $\Pr[y|\mathbf{x}]$?

Optimal Classifier Induced by Conditional Probability

Bayes-optimal classifier

The classifier

$$y \leftarrow \operatorname{argmax}_{y \in \mathcal{Y}} \Pr[y|\mathbf{x}]$$

is optimal over all possible classifiers.

- ▶ $\Pr[y|\mathbf{x}]$ is called **a-posteriori probability** of y .
- ▶ Implication: **Compute $\Pr[y|\mathbf{x}]$ for optimal classification.**
- ▶ How can we know $\Pr[y|\mathbf{x}]$?
- ▶ Suppose we have training data

$$\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$$

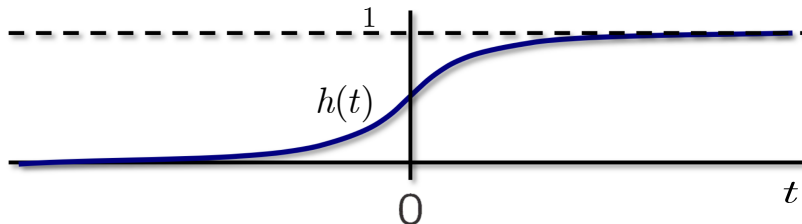
- ▶ We can learn an estimator $\Pr_{\theta}[y|\mathbf{x}]$ for $\Pr[y|\mathbf{x}]$ based on the training data.

Logistic Function / Sigmoid

The function

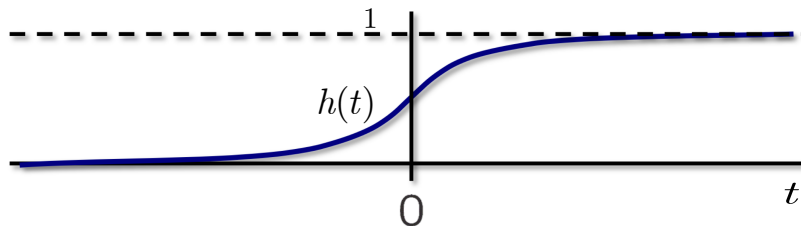
$$h(t) = \frac{e^t}{e^t + 1} = \frac{1}{1 + e^{-t}}$$

is called the **logistic function** or **sigmoid**.



Sigmoid: 'S'-like function.

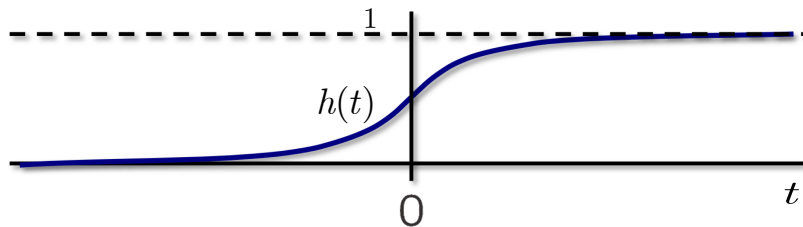
Logistic Function: Probability Interpretation



- ▶ $h(t) \in [0, 1]$ — can be interpreted as probability.
- ▶ $\Pr[y|\mathbf{x}]$ is also a kind of probability.

Link?

Logistic Function: Probability Interpretation



- ▶ $h(t) \in [0, 1]$ — can be interpreted as probability.
- ▶ $\Pr[y|\mathbf{x}]$ is also a kind of probability.

Link? Using $h(t)$ to approximate $\Pr[y|\mathbf{x}]$.

Logistic Regression Model for Binary Classification

Logistic regression (LR) has the following $\Pr_{\theta} [y|\mathbf{x}]$ for modeling $\Pr [y|\mathbf{x}]$:

$$\Pr_{\theta} [y = +1|\mathbf{x}] = h(\boldsymbol{\theta}^{\top} \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^{\top} \mathbf{x}}}$$

$$\Pr_{\theta} [y = -1|\mathbf{x}] = 1 - \Pr_{\theta} [y = +1|\mathbf{x}] = \frac{1}{1 + e^{\boldsymbol{\theta}^{\top} \mathbf{x}}}$$

Thus,

$$\Pr_{\theta} [y|\mathbf{x}] = \frac{1}{1 + \exp \left(-y \cdot \boldsymbol{\theta}^{\top} \mathbf{x} \right)}$$

Logistic Regression Model for Binary Classification

Logistic regression (LR) has the following $\Pr_{\theta} [y|\mathbf{x}]$ for modeling $\Pr [y|\mathbf{x}]$:

$$\Pr_{\theta} [y = +1|\mathbf{x}] = h(\boldsymbol{\theta}^{\top} \mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^{\top} \mathbf{x}}}$$

$$\Pr_{\theta} [y = -1|\mathbf{x}] = 1 - \Pr_{\theta} [y = +1|\mathbf{x}] = \frac{1}{1 + e^{\boldsymbol{\theta}^{\top} \mathbf{x}}}$$

Thus,

$$\Pr_{\theta} [y|\mathbf{x}] = \frac{1}{1 + \exp(-y \cdot \boldsymbol{\theta}^{\top} \mathbf{x})}$$

- ▶ The learning process is to learn a $\hat{\boldsymbol{\theta}}$ such that $\Pr_{\hat{\boldsymbol{\theta}}} [y|\mathbf{x}]$ approximates the underlying $\Pr [y|\mathbf{x}]$ well (at least on training data).
- ▶ **Logistic regression** is actually a **classification** technique.
- ▶ Intrinsically, it is tailored for **binary classification**, $y \in \{+1, -1\}$.

Logistic Regression is a Linear Classifier

Suppose we have learned θ

$$\Pr_{\theta}[y = +1|\mathbf{x}] = \frac{1}{1 + e^{-\theta^{\top} \mathbf{x}}} > \frac{1}{2} \quad (\text{classify } \mathbf{x} \text{ as class } +1)$$

This is equivalent to

$$e^{-\theta^{\top} \mathbf{x}} < 1$$

This is further equivalent to

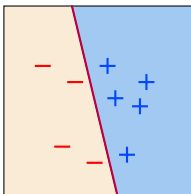
$$\theta^{\top} \mathbf{x} > 0$$

Thus

$$y = \begin{cases} +1, & \theta^{\top} \mathbf{x} > 0 \\ -1, & \theta^{\top} \mathbf{x} < 0 \end{cases}$$

Logistic Regression is a Linear Classifier

$$y = \begin{cases} +1, & \boldsymbol{\theta}^\top \mathbf{x} > 0 \\ -1, & \boldsymbol{\theta}^\top \mathbf{x} < 0 \end{cases}$$



- ▶ This reduces to our linear classification model $f_{\boldsymbol{\theta}}(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x}$.
- ▶ LR and the perceptron are two different methodologies for learning $f_{\boldsymbol{\theta}}(\mathbf{x})$.
- ▶ In LR, How to choose $f_{\boldsymbol{\theta}}(\mathbf{x})$ from $\mathcal{H}$?

Logistic Regression

- Recall training data pairs:

$$\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$$

- Represent a-posteriori probability for $(\mathbf{x}_i, y_i)$

$$\Pr_{\theta} [y_i | \mathbf{x}_i] = \frac{1}{1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^{\top} \mathbf{x}_i \right)}$$

- **Observation:** The **likelihood** of $(\mathbf{x}_i, y_i)$ given parameter $\boldsymbol{\theta}$.

How to learn parameter $\boldsymbol{\theta}$?

Logistic Regression

- Recall training data pairs:

$$\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$$

- Represent a-posteriori probability for $(\mathbf{x}_i, y_i)$

$$\Pr_{\theta} [y_i | \mathbf{x}_i] = \frac{1}{1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^{\top} \mathbf{x}_i \right)}$$

- **Observation:** The **likelihood** of $(\mathbf{x}_i, y_i)$ given parameter $\boldsymbol{\theta}$.

How to learn parameter $\boldsymbol{\theta}$?

Maximum likelihood estimation principle.

Logistic Regression: The Learning Problem

- The likelihood of all data $\{(\mathbf{x}_i, y_i)\}$ (i.i.d.):

$$\prod_{i=1}^n \Pr_{\boldsymbol{\theta}} [y_i | \mathbf{x}_i]$$

- The log-likelihood of all data $\{(\mathbf{x}_i, y_i)\}$:

$$\sum_{i=1}^n \log (\Pr_{\boldsymbol{\theta}} [y_i | \mathbf{x}_i]) = - \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^{\top} \mathbf{x}_i \right) \right)$$

Logistic Regression: The Learning Problem

- ▶ The likelihood of all data $\{(\mathbf{x}_i, y_i)\}$ (i.i.d.):

$$\prod_{i=1}^n \Pr_{\boldsymbol{\theta}} [y_i | \mathbf{x}_i]$$

- ▶ The log-likelihood of all data $\{(\mathbf{x}_i, y_i)\}$:

$$\sum_{i=1}^n \log (\Pr_{\boldsymbol{\theta}} [y_i | \mathbf{x}_i]) = - \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^{\top} \mathbf{x}_i \right) \right)$$

- ▶ Maximum likelihood estimation leads to the learning problem:

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} \quad \frac{1}{n} \sum_{i=1}^n \underbrace{\log \left(1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^{\top} \mathbf{x}_i \right) \right)}_{\ell(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)}$$

What we are going to minimize? Training error measured by **logistic loss**, sometimes also called **cross-entropy** loss. Also related to minimizing in-sample error E_{in} with 0-1 error measure.

Revisiting Generalization: How to Make Er_{out} Small

- Generalization theory says:

$$\forall f_{\boldsymbol{\theta}} \in \mathcal{H} \quad \text{Er}_{\text{out}}(f_{\boldsymbol{\theta}}) \leq \text{Er}_{\text{in}}(f_{\boldsymbol{\theta}}) + \mathcal{O}\left(\sqrt{\frac{d_{\text{VC}}}{n}}\right).$$

- The goal: Make Er_{out} small.
- The generalization error is fixed when $\mathcal{H}$ and training data are fixed.
- Make the $\text{Er}_{\text{in}}(f_{\boldsymbol{\theta}})$ small by choosing a specific $f_{\boldsymbol{\theta}} \in \mathcal{H}$.

How?

Revisiting Generalization: How to Make Er_{out} Small

- Generalization theory says:

$$\forall f_{\theta} \in \mathcal{H} \quad \text{Er}_{\text{out}}(f_{\theta}) \leq \text{Er}_{\text{in}}(f_{\theta}) + \mathcal{O}\left(\sqrt{\frac{d_{\text{VC}}}{n}}\right).$$

- The goal: Make Er_{out} small.
- The generalization error is fixed when $\mathcal{H}$ and training data are fixed.
- Make the $\text{Er}_{\text{in}}(f_{\theta})$ small by choosing a specific $f_{\theta} \in \mathcal{H}$.

How? Design **algorithm** for **training** to pick a $\hat{\theta}$ such that:

$$\min_{\theta \in \mathbb{R}^d} \text{Er}_{\text{in}}(f_{\theta}) \leftarrow \hat{\theta} = \underset{\theta \in \mathbb{R}^d}{\text{argmin}} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(\mathbf{x}_i), y_i).$$

Learned model: $f_{\hat{\theta}} \in \mathcal{H}$, provides small $\text{Er}_{\text{out}}(f_{\hat{\theta}})$.

↪ Gives the motivation for formulating the logistic regression problem.

Logistic Regression vs. Perceptron

- ▶ Perceptron: Find any linear classifier that correctly classify $+1$'s and -1 's, i.e., $\text{sign}(\boldsymbol{\theta}^\top \boldsymbol{x})$ is correct.
- ▶ Logistic Regression: Tend to **simultaneously** classify $+1$'s and -1 's into its right-most and left-most sides, respectively.
- ▶ In addition, logistic regression does not assume linearly separable data.

LR is intuitively better compared to perceptron.

Logistic Regression vs. Least Squares

- ▶ Logistic regression: **logistic loss**

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^\top \mathbf{x}_i \right) \right).$$

- ▶ Least squares: **squared ℓ_2 -norm loss**

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2.$$

Logistic Regression vs. Least Squares

- ▶ Logistic regression: **logistic loss**

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^\top \mathbf{x}_i \right) \right).$$

- ▶ Least squares: **squared ℓ_2 -norm loss**

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2.$$

Optimization

- ▶ Least squares: Closed-form solution.
- ▶ Logistic regression: **No** closed-form.

Logistic Regression vs. Least Squares

- ▶ Logistic regression: **logistic loss**

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \cdot \boldsymbol{\theta}^\top \mathbf{x}_i \right) \right).$$

- ▶ Least squares: **squared ℓ_2 -norm loss**

$$\hat{\boldsymbol{\theta}} = \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2.$$

Optimization

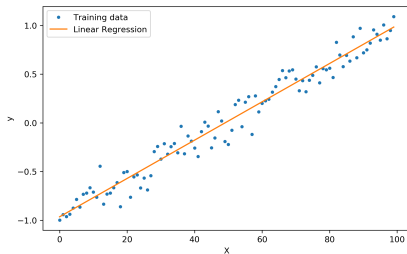
- ▶ Least squares: Closed-form solution.
- ▶ Logistic regression: **No** closed-form.

Regression vs. classification

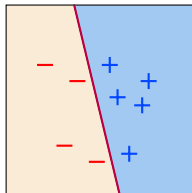
- ▶ Least squares: Tailored for Regression.
- ▶ Logistic regression: Tailored for classification.

Recall: Regression v.s. Classification

Regression



Classification



- ▶ **Regression** is to fit a **continuous** quantity, $y \in \mathbb{R}$ is continuous.
- ▶ **Classification** is to fit a **discrete** labels, $y \in \{-1, +1\}$ is categorical.

Extension: Multi-class Logistic Regression

Softmax: Extension of Logistic Function

- ▶ The logistic regression developed so far is for binary classification.

How about when number of classes $K > 2$?

- ▶ The key idea is to assign each class $k = 1, \dots, K$ a parameter / weight vector θ_k .
- ▶ Let $\Theta = [\theta_1, \dots, \theta_K] \in \mathbb{R}^{(d+1) \times K}$ and $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ be the training data.
- ▶ The model for estimating the a-posteriori of y_i is given by

$$\Pr_{\Theta} [y_i = k | \mathbf{x}_i] = \frac{\exp(\theta_k^{\top} \mathbf{x}_i)}{\sum_{j=1}^K \exp(\theta_j^{\top} \mathbf{x}_i)}$$

also known as **softmax**. It is clear that $\Pr [y_i = k | \Theta, \mathbf{x}_i]$ sum to 1 over k .

Multi-class Logistic Regression

Using the reasoning of MLE, we can formulate the learning problem as

$$\hat{\Theta} = \underset{\Theta \in \mathbb{R}^{d \times K}}{\operatorname{argmin}} -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K 1_{\{y_i=k\}} \log \left(\frac{\exp(\boldsymbol{\theta}_k^\top \mathbf{x}_i)}{\sum_{j=1}^K \exp(\boldsymbol{\theta}_j^\top \mathbf{x}_i)} \right),$$

where $1_{\{y_i=k\}}$ is the indicator function defined as

$$1_{\{y_i=k\}} = \begin{cases} 1, & y_i \text{ is } k\text{-th class} \\ 0, & \text{otherwise} \end{cases}$$

Multi-class Logistic Regression

Using the reasoning of MLE, we can formulate the learning problem as

$$\hat{\Theta} = \underset{\Theta \in \mathbb{R}^{d \times K}}{\operatorname{argmin}} -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K 1_{\{y_i=k\}} \log \left(\frac{\exp(\boldsymbol{\theta}_k^\top \mathbf{x}_i)}{\sum_{j=1}^K \exp(\boldsymbol{\theta}_j^\top \mathbf{x}_i)} \right),$$

where $1_{\{y_i=k\}}$ is the indicator function defined as

$$1_{\{y_i=k\}} = \begin{cases} 1, & y_i \text{ is } k\text{-th class} \\ 0, & \text{otherwise} \end{cases}$$

- Why MLE leads to such a formulation? (HW2).

Summary of Logistic Regression

- ▶ The most important concept in LR is to use logistic function / softmax to approximate $\Pr[y|\mathbf{x}]$, i.e.,

$$\Pr[y|\mathbf{x}] \leftarrow \Pr_{\boldsymbol{\theta}} [y|\mathbf{x}] = \frac{1}{1 + \exp\left(-y \cdot \boldsymbol{\theta}^\top \mathbf{x}\right)}.$$

- ▶ LR is to use the data $\{(\mathbf{x}_i, y_i)\}$ directly to learn such a model $\Pr_{\boldsymbol{\theta}} [y|\mathbf{x}]$.
- ▶ Later, we will study that **deep neural networks** and **language models** are also learning this model $\Pr_{\boldsymbol{\theta}} [y|\mathbf{x}]$ but not directly using the data $\{(\mathbf{x}_i, y_i)\}$. $\rightsquigarrow$ Later lectures on deep learning.

How to Learn $\hat{\theta}$?

The objective function is (using binary logistic regression as an example):

$$\mathcal{L}(\theta) := \frac{1}{n} \sum_{i=1}^n \log \left(1 + \exp \left(-y_i \cdot \theta^\top x_i \right) \right)$$

The learning problem is formulated as

$$\hat{\theta} = \underset{\theta \in \mathbb{R}^d}{\operatorname{argmin}} \mathcal{L}(\theta)$$

- ▶ Bad news $\times$: No closed-form solution.
- ▶ Good news $\checkmark$: The objective function $\mathcal{L}(\theta)$ is **convex** in θ .

$\rightsquigarrow$ Next lectures: Convex optimization and gradient-based optimization algorithms.